Service Center Job Tracking & Customer Visit Analytics Pipeline

Project Proposal & Technical Overview

Domain: Field Service Operations Analytics	**Type:** Batch Data Engineering Pipeline	**Stack:** Python · PySpark · Databricks · Delta Lake · SQL

1. Real-World Context

Walk into any mobile repair outlet, laptop service center, or automotive workshop, and you will find one thing in common — every interaction generates a paper trail. When a customer walks in with a broken device, a job card is raised. That job card carries a unique ID, records the customer, the device, the reported issue, the assigned technician, and the expected delivery date. When the device is handed back, the card is closed.

This data is not collected for analytics — it is collected because the business cannot function without it. Job tracking is how technicians know what to work on. Status updates are how front-desk staff answer customer calls. Delivery dates are how the center manages customer expectations. The data already exists, in every service center, every day.

The problem is that this data is almost never used beyond its immediate operational purpose. Once a job is closed and the device delivered, the card is filed away and forgotten. The patterns buried in months of job records — which technicians are slower, which devices fail most often, which customers keep coming back — are invisible.

This project builds the pipeline that makes those patterns visible.

2. Problem Statement

"Service centers collect job and customer data every single day as a mandatory operational requirement. But they lack the structured analytics infrastructure to turn that data into actionable insight — on workload distribution, repair delays, technician performance, repeat customer patterns, and device failure trends."

In concrete terms, a typical service center manager cannot currently answer:

- Which technician closes jobs the fastest — and which consistently runs over the promised delivery date?
- How many of this month's customers are returning for the same recurring issue?
- What percentage of jobs are delayed, and by how many days on average?
- Which device brand generates the highest volume of repair jobs?
- Are certain issue types being resolved faster than others, or are specific fault categories bottlenecks?

These questions have answers sitting in the job records. The gap is not data — it is the absence of a pipeline to structure, enrich, and surface that data for decision-making.

3. What This Project Solves

This pipeline ingests raw service job records, cleans and enriches them, and delivers a structured set of business metrics that the operations and management teams can query directly. Every insight the pipeline produces is grounded in data the service center already generates — nothing is fabricated, no external data source is required.

The pipeline answers five operational questions every service center should have data for:

- **Repair Turnaround Time —** How long does each repair actually take, relative to what was promised?
  - Computed per job by deriving the gap between job creation and job completion. Flagged as Delayed or On Time based on configurable thresholds.

- **Technician Performance —** Who are the highest-performing technicians, and who needs support?
  - Average repair time per technician surfaces relative efficiency. Outliers become visible — both the fast closers and the consistent bottlenecks.

- **Repeat Customer Detection —** Which customers are returning repeatedly for the same type of issue?
  - Customers with more than one job record are identified. Repeat visits for the same fault category indicate either a quality problem or a recurring hardware failure.

- **Device & Fault Trend Analysis —** Which device brands and issue types drive the highest job volume?
  - Aggregated job counts by brand, device type, and issue category reveal which products are generating the most service demand — useful for both operations planning and supplier feedback.

- **Customer Visit History —** What is the most recent job for each customer, and what was its outcome?
  - Using ranking logic, the latest interaction per customer is extracted cleanly — the starting point for any CRM-style customer view.

4. Dataset — What Data the Pipeline Uses

The pipeline is structured around three tables that reflect how service centers actually store their operational data. These are not contrived academic datasets — this is the exact data model used in real service CRM systems.

The dataset provided contains 300 customer records, 43 device records, and 1,510 service job records (including intentional duplicates and null values) spanning a 90-day operational window from January to March 2024.

4a. Customers (customers.csv — 300 rows)

A record per unique customer. Every job can be traced back to a customer record through the customer_id foreign key. This table enables repeat visit analysis and customer-level history queries.

Column Name	Data Type	Nullable	Description
customer_id	VARCHAR	NOT NULL	Primary key. Unique identifier in format CUST0001 – CUST0300.
customer_name	VARCHAR	NOT NULL	Full name of the customer (first + last).
phone_number	VARCHAR	NOT NULL	10-digit mobile number. Unique per customer.
email	VARCHAR	NOT NULL	Customer email address. Unique per customer.
city	VARCHAR	NOT NULL	City of residence (14 Indian cities represented).
registration_date	DATE	NOT NULL	Date the customer first registered with the service center.

4b. Devices (devices.csv — 43 rows)

One record per device category and brand combination. Linked to service jobs via device_id, this table lets the pipeline aggregate failure trends by type — for example, whether a particular laptop brand drives disproportionately high repair volume.

Column Name	Data Type	Nullable	Description
device_id	VARCHAR	NOT NULL	Primary key. Format DEV001 – DEV043.
brand	VARCHAR	NOT NULL	Device manufacturer (e.g. Samsung, Apple, Dell, Xiaomi).
device_type	VARCHAR	NOT NULL	Category of device (e.g. Smartphone, Laptop, Air Conditioner).
model_series	VARCHAR	NOT NULL	Brand + device type short label (e.g. Samsung SMA-Series).
warranty_months	INTEGER	NOT NULL	Standard warranty duration in months: 6, 12, 18, or 24.
price_range	VARCHAR	NOT NULL	Approximate price tier: Budget / Mid-range / Premium.

4c. Service Jobs (service_jobs.csv — 1,510 rows)

The core transaction table. Every job record carries its own identifier and references both a customer and a device. This single table is the foundation of every metric the pipeline produces. The dataset intentionally includes 10 duplicate rows and approximately 75 records with a blank technician_name — both must be handled in the pipeline.

Column Name	Data Type	Nullable	Description
job_id	VARCHAR	NOT NULL	Unique job identifier in format JOB00001. 10 duplicate rows exist intentionally — must be de-duplicated.
customer_id	VARCHAR	NOT NULL	Foreign key referencing customers.customer_id.
device_id	VARCHAR	NOT NULL	Foreign key referencing devices.device_id.
issue_type	VARCHAR	NOT NULL	Type of fault reported (e.g. Screen Damage, Battery Issue, Overheating).
job_status	VARCHAR	NOT NULL	Current status: Completed | In Progress | Pending | Cancelled.
received_date	DATE	NOT NULL	Date the device was received at the service center.
promised_date	DATE	NOT NULL	Committed delivery date given to customer (received_date + 5 days SLA).
completed_date	DATE	NULLABLE	Actual completion date. NULL for In Progress and Pending jobs.
technician_id	VARCHAR	NOT NULL	ID of assigned technician: T001 – T008.
technician_name	VARCHAR	NULLABLE	Name of assigned technician. ~75 rows intentionally blank — must be handled.
repair_notes	VARCHAR	NULLABLE	Free-text notes on repair status or outcome. ~8% blank.
estimated_cost	DECIMAL	NOT NULL	Initial cost estimate provided to customer (INR).
actual_cost	DECIMAL	NULLABLE	Actual amount charged on completion. NULL for non-Completed jobs.

Intentional data quality issues in service_jobs.csv: 10 duplicate job_id rows | ~75 blank technician_name values | ~8% blank repair_notes | completed_date NULL for non-completed jobs | actual_cost NULL for non-completed jobs

5. Pipeline Architecture — Medallion Design

The pipeline is structured around the Medallion Architecture — a three-layer data design pattern that is the standard approach for building reliable, auditable analytical systems on modern data lake platforms. Data moves through three layers, each with a clearly defined role and quality standard.

Bronze Layer — Raw Ingestion

Raw job data arrives as CSV exports and is loaded into the Bronze layer without modification. Every field is preserved exactly as received — including nulls, duplicates, inconsistencies, and formatting issues present in the source. The Bronze layer is an immutable record of what came in. If a downstream processing step introduces an error, Bronze is the clean starting point for reprocessing, with no need to go back to the source files.

Silver Layer — Cleaning & Enrichment

The Silver layer is where data quality is established and business logic is first applied. Date fields are parsed and typed correctly. Duplicate job records are removed. Blank technician names are resolved using the technician_id. The most important enrichment at this stage is the computation of repair duration — calculated as the difference in days between received_date and completed_date. This derived field is the foundation for every turnaround and delay analysis downstream. At this layer, the three source tables — customers, devices, and service jobs — are joined, producing a single enriched record per job that carries all the context needed for analytics.

Gold Layer — Business Metrics

The Gold layer is pre-aggregated and structured for the specific questions the business needs answered. Separate aggregated tables are materialised for technician performance, delay analysis, device brand trends, and customer repeat visit counts. This layer is designed for speed and accessibility — a stakeholder querying the Gold layer should get an answer in seconds, not minutes, and should not need to understand the underlying data model to do so.

6. Technology Stack & Justification

Python

Python handles the data generation and ingestion scripting layer. In a live deployment, the same Python layer would serve as the connector that pulls job records from the operational CRM system into the pipeline on a scheduled basis.

Apache PySpark on Databricks

PySpark drives all transformations across the three pipeline layers. Its distributed processing model means the pipeline is built for scale from the start — the same code that processes 1,500 job records in development will process five million records in a national service chain deployment without modification. Spark's lazy evaluation model is particularly valuable here: all transformation steps are assembled as a logical plan, and the Catalyst Optimizer determines the most efficient physical execution path before any data is processed. This optimisation becomes significant when multi-table joins, window functions, and multiple aggregation passes are combined in a single pipeline run.

Databricks provides the managed Spark runtime, removing the need to provision or maintain cluster infrastructure. The notebook-based development environment on Databricks also makes the pipeline easy to review, share, and iterate on collaboratively.

Delta Lake

Delta Lake is the storage layer used across all three pipeline tiers. It adds a transaction log on top of Parquet-based storage, enabling capabilities that are critical for a production pipeline: ACID guarantees ensure that a failed write never leaves a table in a partially written, corrupted state. Time travel allows any table to be queried as it appeared at a prior point — enabling rollback if a bad transformation reaches Gold before it is caught. Schema enforcement ensures that any record deviating from the expected structure is rejected at write time rather than silently corrupting the dataset.

For technician details, Delta Lake's MERGE operation enables SCD Type 2 tracking — preserving the full history of any technician attribute changes so that historical job records remain accurate even after technician data is updated. This is the correct approach for any dimension that can change over time in a production data warehouse.

SQL — Analytics Layer

SQL is the final interface between the engineering pipeline and the people who need to use its output. Once Gold layer tables are registered in the Databricks metastore, analysts, operations managers, and business stakeholders can query them directly — without any knowledge of the underlying Python or Spark code.

The analytics built on top of the Gold layer demonstrate the full range of SQL capabilities:

- CASE expressions to classify jobs as Delayed or On Time based on repair duration thresholds.
- GROUP BY with HAVING filters to identify customers with more than one recorded visit — the repeat customer signal.
- Window functions using PARTITION BY to compute each technician's average repair time alongside individual job records — enabling precise comparison.
- Common Table Expressions (CTEs) with ROW_NUMBER to extract the most recent job per customer cleanly, without subquery nesting.
- Multi-table joins at the Silver layer to produce a single enriched fact record per job from the three source tables.

7. Business Value & Operational Impact

The insights this pipeline produces are directly actionable by a service center operations team. They do not require data science expertise to interpret, and they address questions that managers in this industry face every day.

- **Technician Performance Management —** Identify which technicians are consistently slower than their peers. Use this to calibrate training, workload distribution, or mentoring assignments — rather than relying on anecdotal observation.

- **Quality Assurance —** Flag repeat customers arriving with the same fault category. If a device type is repeatedly failing for the same reason, this signals either a quality issue with the device brand or an incomplete fix the first time — both of which have reputational and warranty implications.

- **Delay Reduction —** Surface the proportion of jobs that exceed the promised turnaround time. A center with a 30-percent delay rate can reduce it through better workload planning. Without the data, the problem is invisible.

- **Operational Planning —** Understand which device brands and fault types drive the highest job volume. This informs spare parts procurement, staffing decisions, and — for larger chains — supplier feedback.

- **Customer Experience —** Build a customer-level interaction history that gives front-desk staff context before a customer even states their reason for visiting.

8. Why This Stack Works Together

The technology choices in this project are not independent decisions — they form a coherent architecture where each component is justified by what comes before and after it.

Python handles the boundary between the external operational system and the pipeline. PySpark handles the transformation logic at the scale the business will eventually need. Delta Lake provides the reliability and auditability layer that makes the pipeline trustworthy in production. SQL provides the access layer that makes the pipeline's output usable by people who are not engineers.

The Medallion Architecture is the structural principle that ties these components together. Bronze preserves the source. Silver establishes quality and adds business context. Gold serves the stakeholder. Each layer has a single, clear responsibility, which makes the pipeline easy to understand, debug, extend, and hand off. This is what a production-grade data engineering system looks like.

Dataset included: customers.csv (300 rows) | devices.csv (43 rows) | service_jobs.csv (1,510 rows)

Project proposal prepared for internal evaluation and submission.